



西安电子科技大学
XIDIAN UNIVERSITY

《操作系统》

第2章 进程管理

Process Management

人工智能学院

沟通与同步的重要意义



冰丝带



雪如意

北京冬奥会场馆建设（大型工程协作）

- ◆ 进程通信
- ◆ 进程间相互作用

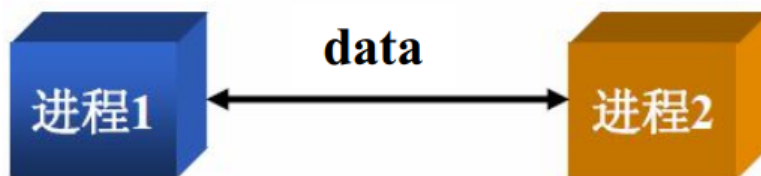
进程通信

进程通信：简单来说就是在进程间传输数据(交换信息)。

进程通信的分类

❖ 根据通信方式不同，分为：

➤ **直接通信**：信息直接传递给接收方。



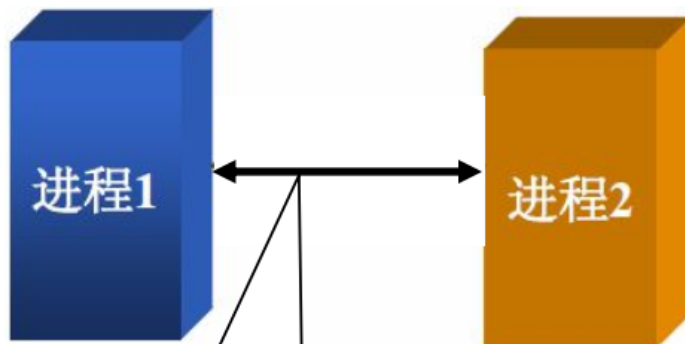
➤ **间接通信**：借助于收发双方进程之外的共享数据结构作为通信中转。



进程通信

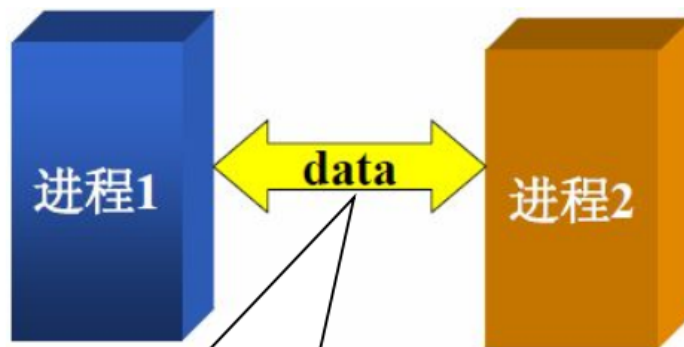
❖ 根据交换信息量的多少和效率的高低，分为：

低级通信



只能传递状态和整数值
(控制信息)，传送信息
量小。

高级通信



能够传递大量数据，通信效率高。

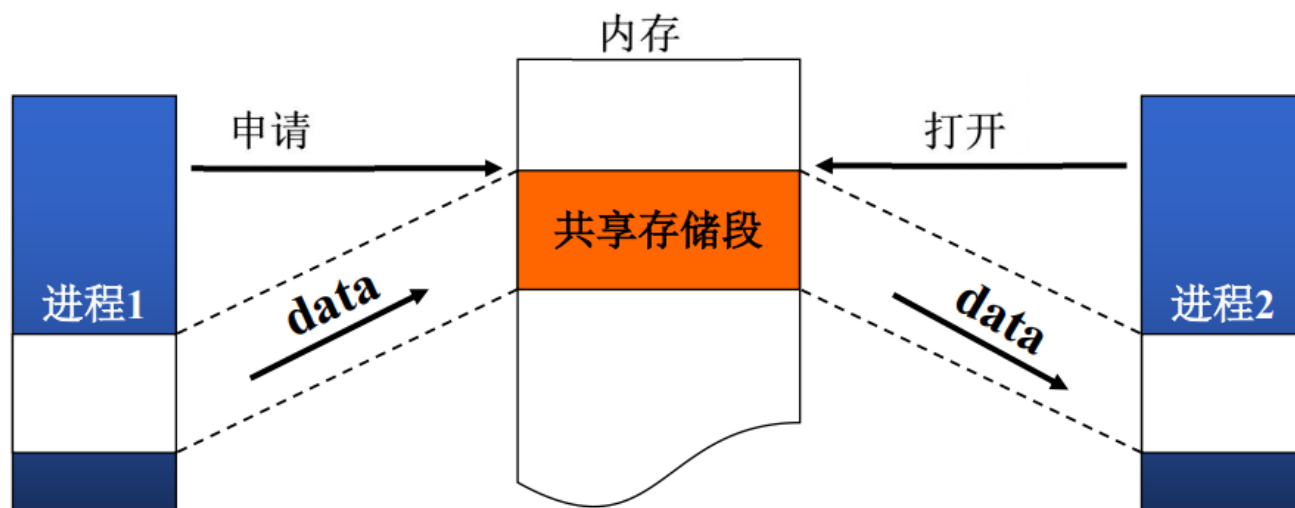
系统提供的高级通信方式：

- (1) 共享内存模式；
- (2) 消息传递模式；
- (3) 共享文件模式。

进程通信

共享内存模式

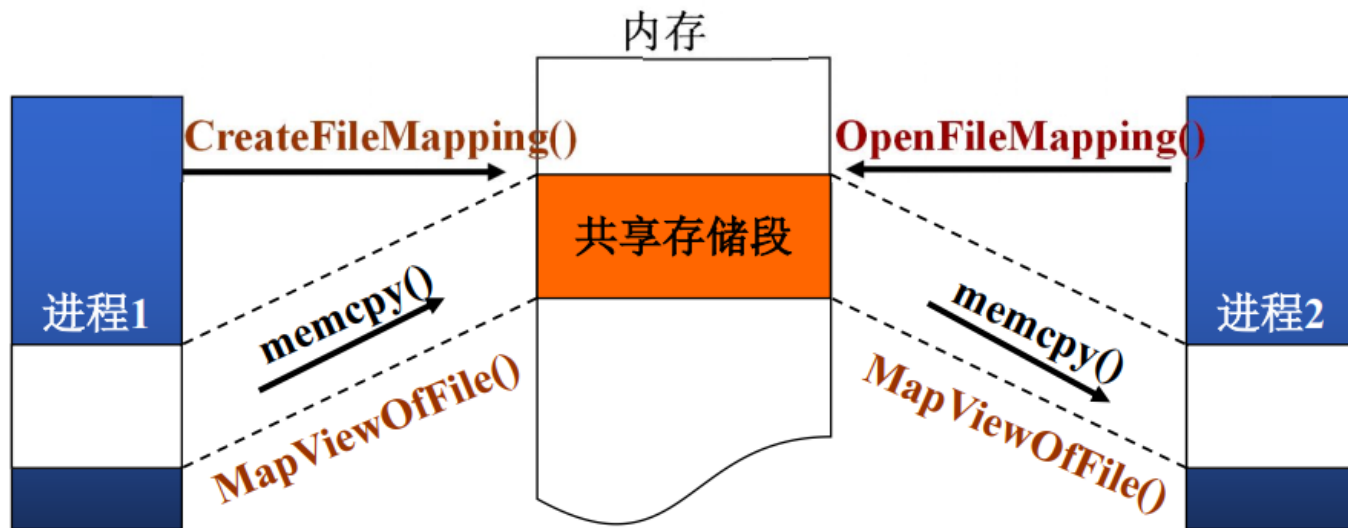
- ❖ 最为快捷有效的方式之一。
- ❖ 间接通信，原理：



- 系统本身不提供对内存共享存储区的互斥控制，因此互斥要通过其它机制实现；
- 数据的发送方不关心数据由谁接收，数据的接收方也不关心数据是由谁发送的，存在安全隐患。

Windows中共享内存模式的实现方式(API函数)

- ❖ **CreateFileMapping()**: 函数创建一个内存映射文件对象
- ❖ **MapViewOfFile()**: 将文件映射对象的视图映射进地址空间
- ❖ **memcpy()**: 数据复制到共享内存
- ❖ **OpenFileMapping()**: 打开一个命名的共享文件对象



进程通信

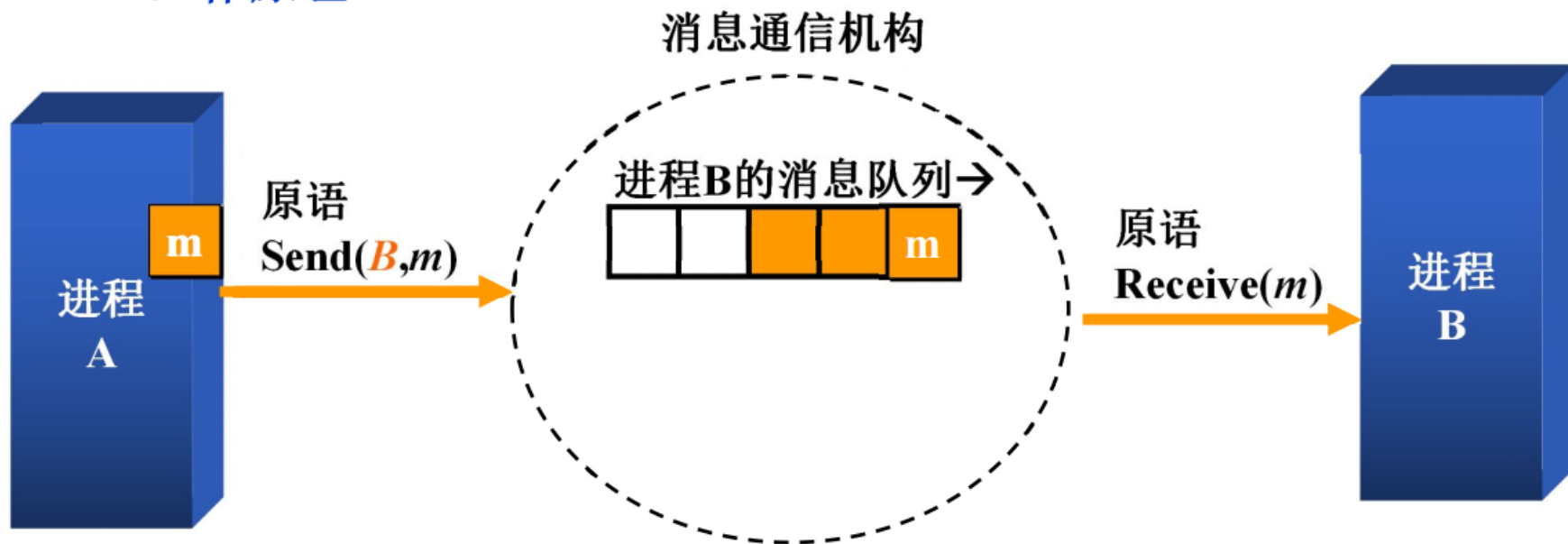
消息传递模式

- ❖ 消息(message)的概念：由发送方形成，通过一定的机制传递给接收方的一组信息，它的长度可以固定，也可以变化。
- ❖ 消息的组成：



进程通信

❖ 工作原理



进程通信

❖ 消息传递的方式

- 直接通信方式：点到点的发送。

Send (*DestProcessName*, *Message*);

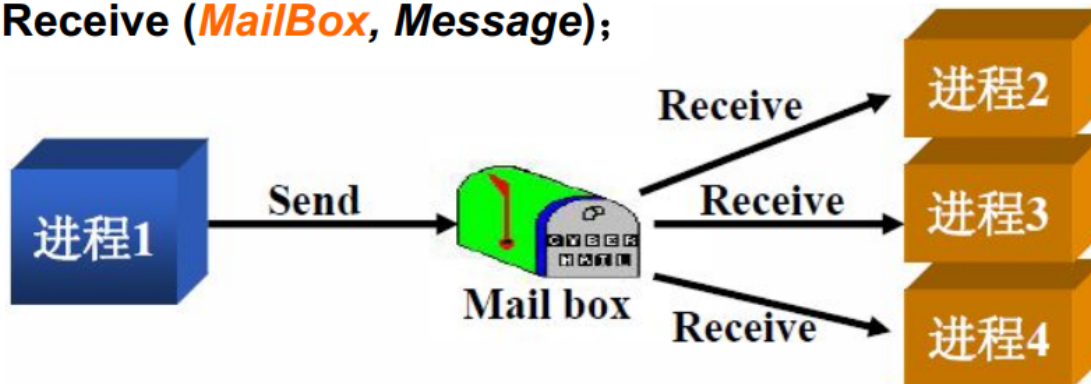
Receive (*SourceProcessName*, *Message*);



- 间接通信方式：以信箱为媒介进行传递，可以广播。

Send (*MailBox*, *Message*);

Receive (*MailBox*, *Message*);



进程通信

管道(共享文件模式)

- ❖ 管道是一种**信息流**缓冲机构，基于文件系统，可以连接两个进程，以先进先出(**FIFO**)方式的单向传送数据。
- ❖ 匿名管道(**Anonymous Pipes**)
 - 匿名管道是一种未命名的、单向管道。通常用来在父进程和子进程之间传输数据。**匿名管道总是本地的，不能在网络之间传递数据。**
- ❖ 命名管道(**Named Pipes**)
 - 命名管道是一种有名称的，可在同一台计算机的不同进程之间或跨越一个网络的不同计算机的不同进程之间传输数据的进程通信方式，支持可靠的，单向或双向的数据通信。

★ 操作系统中的父进程和子进程的概念

父进程：一个创建了另一个新进程的现有进程，它是创造者

子进程：被父进程新创建出来的进程，它是被创造者

父进程通过一个叫做 **fork()** 的系统调用来创建子进程

复制：当父进程调用 `fork()` 时，操作系统会复制父进程的几乎所有内容，创建一个全新的进程，这就是子进程。

复制的内容包括：代码段（程序指令）、数据段（变量、内存内容）、堆栈段、文件描述符表（这意味着父进程打开的文件，子进程也能操作）、当前的工作目录、环境变量等。

进程通信

★ 操作系统中的父进程和子进程的概念

父进程：一个创建了另一个新进程的现有进程，它是创造者

子进程：被父进程新创建出来的进程，它是被创造者

创建后——`exec()` 系列调用

刚创建的子进程是父进程的一个“克隆体”。但大多数时候，**创建子进程是为了让它去执行一个全新的、不同的程序**。这时就需要用到 `exec()` 系列调用（如 `execlp`, `execvp` 等）

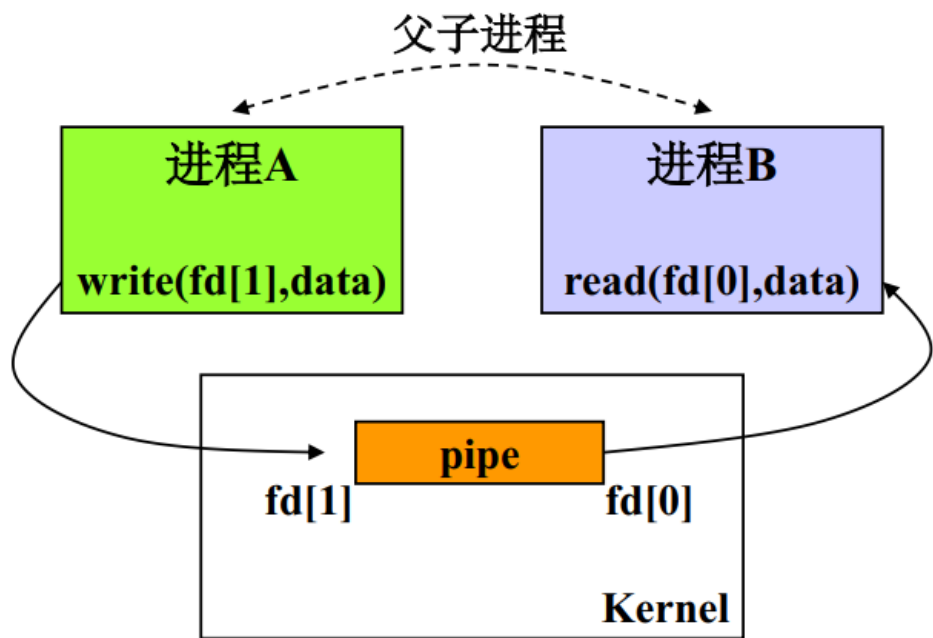
- `exec()` 会用一个新的程序完全替换当前进程（子进程）的内存空间（代码、数据、堆栈等），然后开始执行这个新程序的 `main` 函数
- 经典模式：父进程先 `fork()` 出一个自己的副本（子进程），然后子进程立即调用 `exec()` 来加载并运行一个新程序（如 `ls`, `grep` 等）

★ 为什么要创建子进程执行新程序？

- **保护自己**：让新程序在子进程这个“沙箱”里运行，即使它崩溃了，父进程也不会受影响，依然稳定
- **并发执行**：父子进程可以同时干活，比如后台运行任务
- **进行控制**：父进程可以等待、管理并获取子进程的结果

进程通信

❖ 原理及实现(匿名管道)



匿名管道的特点:

- 单向通信信道，如果进程间要进行双向通信，通常需要定义两个管道。
- 管道通过系统调用read(), write()函数进行读写操作。

调用pipe()函数创建管道:

```
int fd[2];  
pipe(fd);
```


- ◆ 进程通信
- ◆ 进程间相互作用

进程间的相互作用

无序竞争——混乱和崩溃的推手

```
爱鱼的 欧阳  
i=0  
while(i<10){  
    i++  
}  
Printf“ win”
```

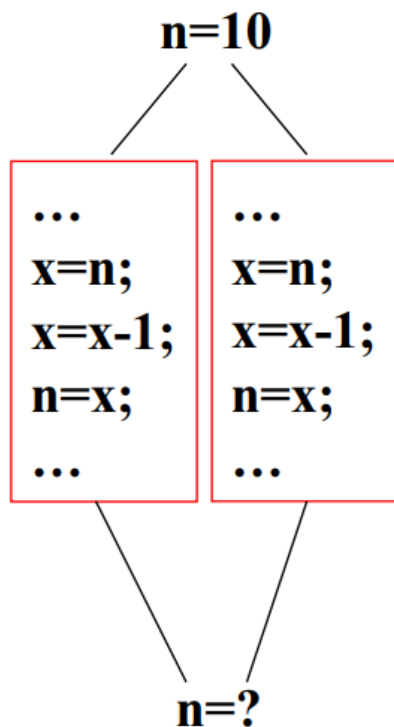
```
爱鱼的 完颜  
i=0  
while(i>-10){  
    i--  
}  
Printf“ win”
```

Who Will Win?

临界资源竞争

进程间的相互作用

★ 并发进程的问题



结果?

可能

进程A: $x=1$;

进程B: $x=2$;

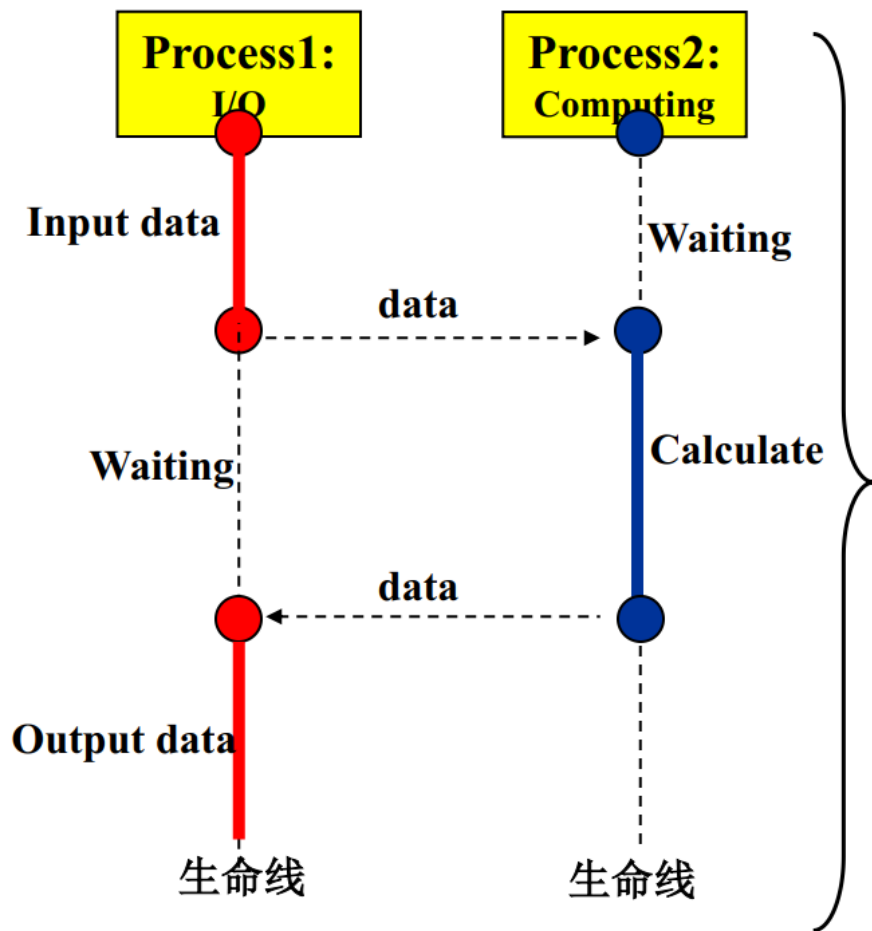
并不确定

临界区 (Critical Section) 问题——多个进程共享资源 (n) 时, 若不限制 “同一时间只有一个进程访问”, 就会因交错执行导致结果错误

如何约束进程的行为, 使其不管如何动态执行, 其结果都能保持正确?

进程间的相互作用

★ 进程同步



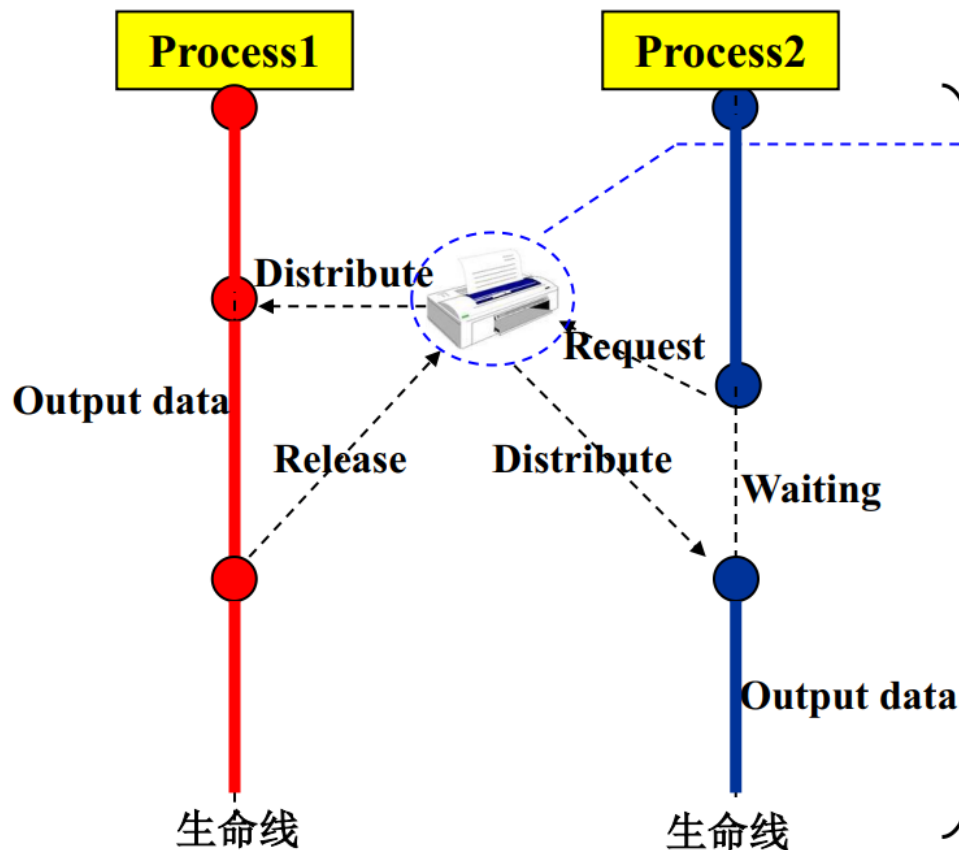
进程之间这种相互合作、协同工作的关系称为进程的**同步**。

简单说来就是：多个相关进程在执行次序上的协调。

制约关系：直接制约。

进程间的相互作用

★ 进程互斥



临界资源：也称独占资源，是指在一段时间内只允许一个进程访问的资源。例如打印机，磁带机，也可以是进程共享的数据、变量等。

多个进程因为争夺**临界资源**而相互排斥执行的过程称为进程的**互斥**。

制约关系：间接制约。

进程间的相互作用

养鱼，锁的前世今生

两人共养一条金鱼，该鱼每天须且仅需进食一顿，不吃饿死、吃多胀死

爱鱼的**欧阳**

```
if (noFeed) {  
    feed fish  
}
```

爱鱼的**完颜**

```
if (noFeed) {  
    feed fish  
}
```

约定喂鱼程序，随意

爱鱼的**欧阳**

T1 观察鱼（没有喂）

T2

T3

T4 喂鱼

R

爱鱼的**完颜**

观察鱼（没有喂）

喂鱼

鱼胀死了!!

没有同步，喂鱼的可能结果

进程间的相互作用

养鱼，锁的前世今生

两人共养一条金鱼，该鱼每天须且仅需进食一顿，不吃饿死、吃多胀死

爱鱼的 欧阳

```
if (noNote) {  
    leave note  
    if (noFeed) {  
        feed fish  
    }  
    Remove note  
}
```

爱鱼的 完颜

```
if (noNote) {  
    leave note  
    if (noFeed) {  
        feed fish  
    }  
    Remove note  
}
```

约定喂鱼程序，留字条

爱鱼的 欧阳

T1 检查发现没有字条
T2
T3
T4 留字条
T5 观察鱼（没有喂）
T6
T7
T8 喂鱼
R 鱼胀死了!!

爱鱼的 完颜

检查发现没有字条
留字条
观察鱼（没有喂）
喂鱼

先检后留（字条），喂鱼的可能结果

进程间的相互作用

养鱼，锁的前世今生

两人共养一条金鱼，该鱼每天须且仅需进食一顿，不吃饿死、吃多胀死

爱鱼的 欧阳

```
if (noNote) {  
    leave note  
    if (noFeed) {  
        feed fish  
    }  
    Remove note  
}
```

爱鱼的 完颜

```
if (noNote) {  
    leave note  
    if (noFeed) {  
        feed fish  
    }  
    Remove note  
}
```

约定喂鱼程序，改进留字条

爱鱼的 欧阳

T1
T2 留字条
T3 检查发现字条（存在）
T4
T5
T6 拿掉字条
R

爱鱼的 完颜

留字条

检查发现字条（存在）
拿掉字条

无人喂鱼，鱼饿死!!

先留后检（字条），喂鱼的可能结果

进程间的相互作用

养鱼，锁的前世今生

两人共养一条金鱼，该鱼每天须且仅需进食一顿，不吃饿死、吃多胀死

爱鱼的**欧阳**

lock ()

if (noFeed) {

 feed fish

}

unlock ()

爱鱼的**完颜**

lock ()

if (noFeed) {

 feed fish

}

unlock ()

约定喂鱼程序，加锁保护，检留、留检一体

- **锁**→操作系统提供的**同步原语**（如互斥锁Mutex），解决“临界资源访问的原子性、互斥性”问题
- **原子化**→把多个微小操作（检查、喂鱼）**打包**成“不可分割”的整体，防止其他进程穿插干扰

进程间的相互作用

锁变量法(测试和设置指令)

- ❖ 做法：设置一个共享变量 W (锁)，初值为 0 。当一个进程想进入其临界区(进程中涉及临界资源的程序段)时，它首先测试这把锁：如果锁的值为 0 ，则进程将其置为 1 并进入临界区。若锁已经为 1 ，则进程等待直到其变成 0 。

- ❖ 实现：

- 加锁原语 $LOCK(W)$: $L: if W=1 then goto L else W=1;$
- 解锁原语 $UNLOCK(W)$: $W=0;$

Process1

```
.....  
LOCK(W); //加锁原语  
临界区;  
UNLOCK(W); //解锁原语  
.....
```

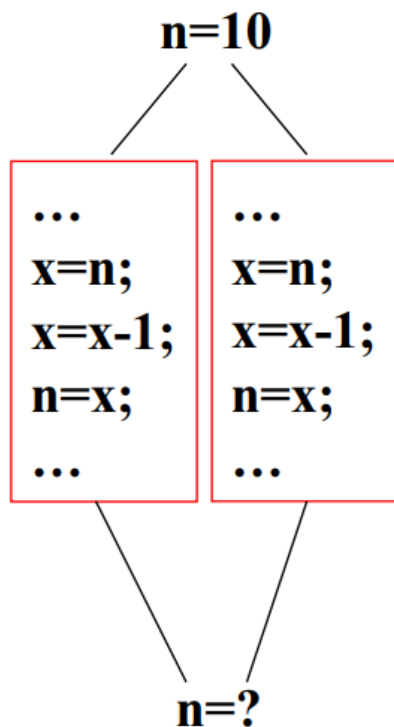
Process2

```
.....  
LOCK(W); //加锁原语  
临界区;  
UNLOCK(W); //解锁原语  
.....
```

现象：CPU会一直处于忙等待状态(Busy waiting)。

回顾——进程间的相互作用

★ 并发进程的问题



结果?

可能

进程A: $x=1;$

进程B: $x=2;$

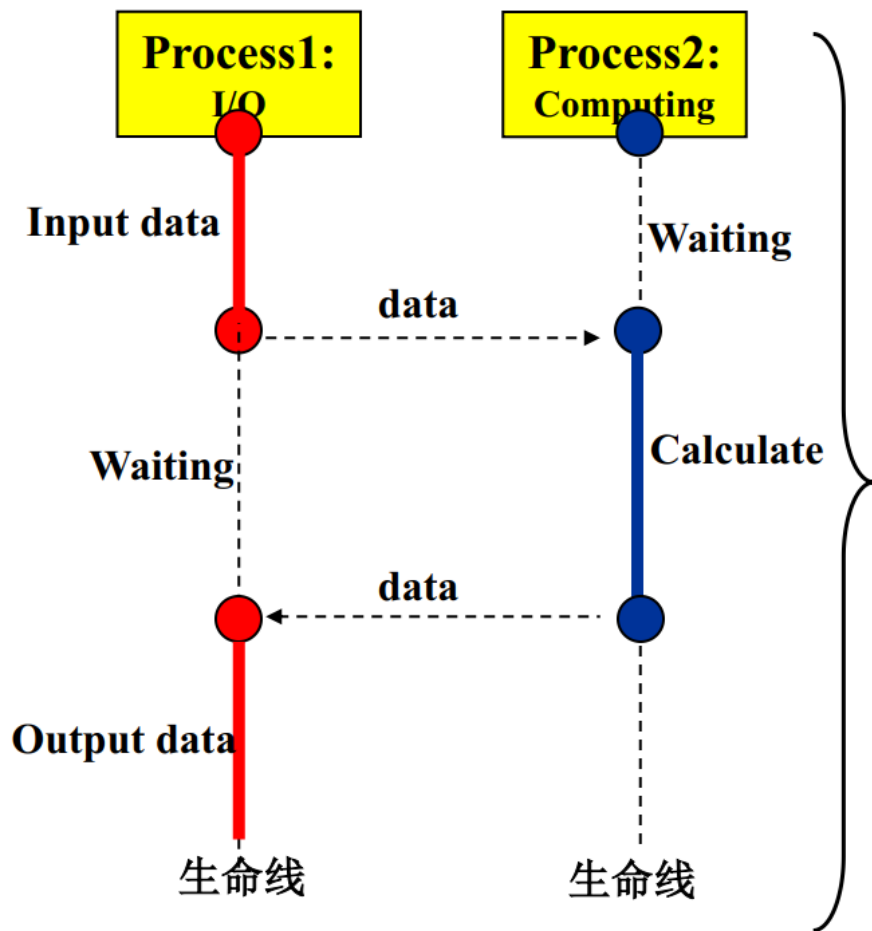
并不确定

临界区 (Critical Section) 问题——多个进程共享资源 (n) 时, 若不限制 “同一时间只有一个进程访问”, 就会因交错执行导致结果错误

如何约束进程的行为, 使其不管如何动态执行, 其结果都能保持正确?

回顾——进程间的相互作用

★ 进程同步



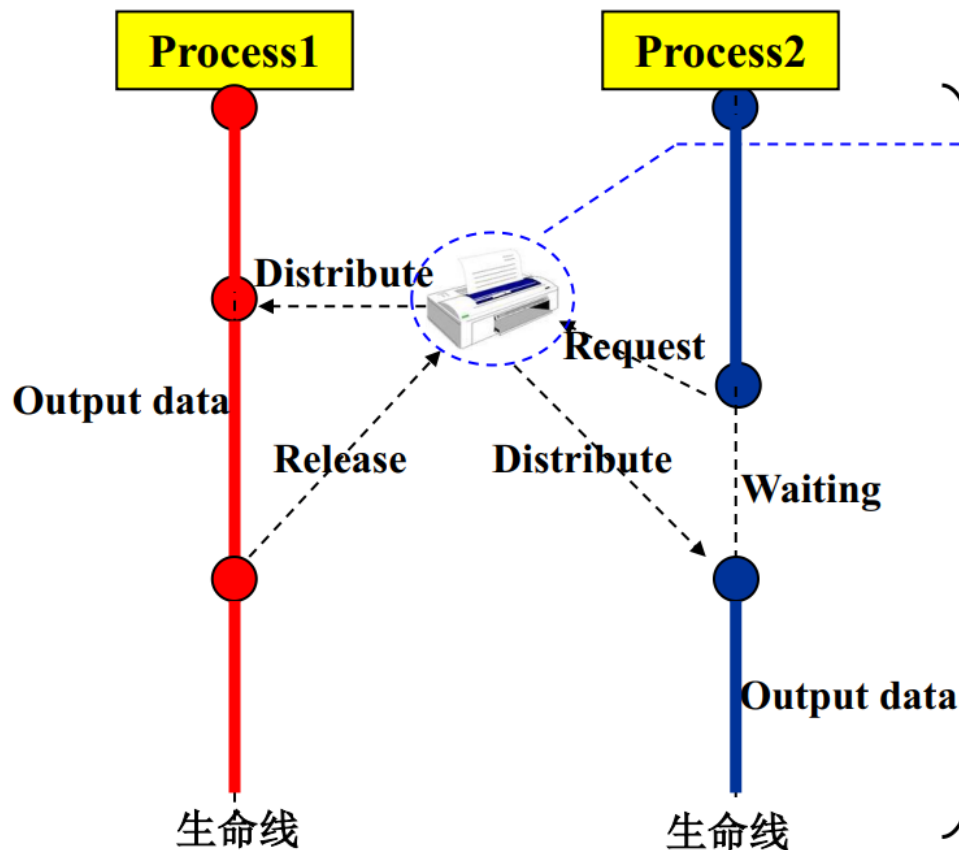
进程之间这种相互合作、协同工作的关系称为进程的**同步**。

简单说来就是：多个相关进程在执行次序上的协调。

制约关系：直接制约。

回顾——进程间的相互作用

★ 进程互斥



临界资源：也称独占资源，是指在一段时间内只允许一个进程访问的资源。例如打印机，磁带机，也可以是进程共享的数据、变量等。

多个进程因为争夺**临界资源**而相互排斥执行的过程称为进程的**互斥**。

制约关系：间接制约。

进程间的相互作用——简单来说

★ 进程同步与互斥

进程同步 (Process Synchronization) - **总目标**: 协调多个进程的执行顺序, 包含:

- **进程互斥 (Process Mutual Exclusion)** - **子目标**: 解决 “竞争” 问题
关键: 禁止同时访问。保证一个临界资源在某一时刻只被一个进程访问。
- **进程协作 (Process Cooperation)** - **子目标**: 解决 “协作” 问题
关键: 控制执行顺序。某个进程的执行需要等待另一个进程提供的结果或信号。

- 把**同步**理解为 “协调次序”, 这个次序既包括了 “谁先谁后” 的严格顺序 (协作), 也包括了 “绝对不能同时” 的独占顺序 (互斥)
- **同步**是**最终要实现的目的**, **互斥**是同步**需要解决的具体问题之一**

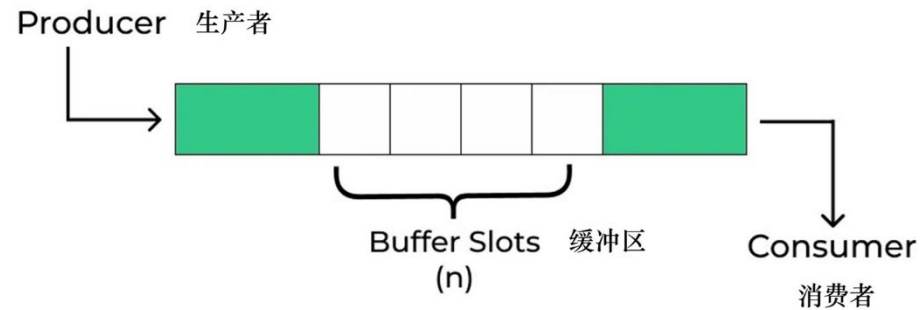
进程间的相互作用

生产者-消费者问题

sleep & wakeup

- 生产者进程

```
void producer(void) {  
    int item;  
    while(True){  
        item = produce_item(); // 1. 生产者生成一个数据项  
        if(count == N) sleep(); // 2. 检查：如果缓冲区满了，就休眠（阻塞）  
        insert_item(item);      // 3. 将数据项放入缓冲区（需被唤醒）  
        count = count + 1;      // 4. 增加缓冲区计数  
        if(count == 1) wakeup(consumer); // 5. 检查：如果放入后缓冲区从空变为非空  
                                     (count 从0->1)，就去唤醒消费者  
    }  
}
```



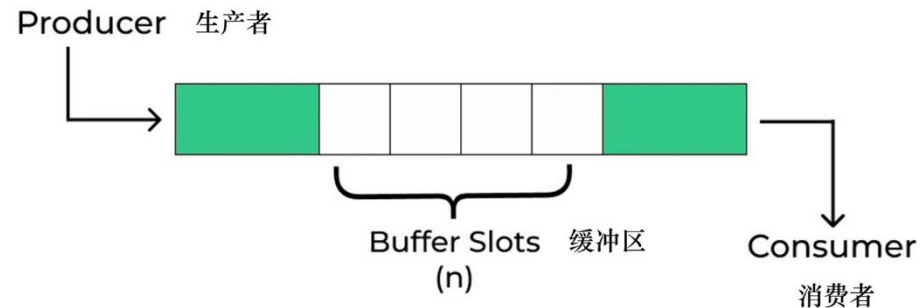
进程间的相互作用

生产者-消费者问题

sleep & wakeup

- 消费者进程

```
void consumer(void) {  
    int item;  
    while(True){  
        if(count == 0) sleep();    // 1. 检查：如果缓冲区为空，就休眠（阻塞）  
        item = remove_item();    // 2. 从缓冲区取出一个数据项（需被唤醒）  
        count = count - 1;    // 3. 减少缓冲区计数  
        if(count == N-1) wakeup(producer); // 4. 检查：如果取出后缓冲区从满变为不满  
                                           (count从N->N-1)，就去唤醒生产者  
        consume_item(item);    // 5. 消费这个数据项  
    }  
}
```



进程间的相互作用

生产者-消费者问题

sleep & wakeup 存在的问题——丢失唤醒

当一个进程用wakeup唤醒另一个进程时，若被唤醒的进程没及时响应（比如还没进入阻塞态），唤醒信号就会“丢失”，导致进程永远阻塞，同步逻辑崩坏

假设：缓冲区满（ $\text{count} == N$ ），生产者执行sleep()，进入阻塞；消费者取走1条鱼，count变为N-1，执行wakeup(producer)；

但如果生产者还没真正进入阻塞态（比如CPU调度延迟），这时候wakeup发的信号就没人收，生产者会一直睡，永远不生产→丢失唤醒

根本原因：对全局变量count的检查和修正是分离的，不是原子操作。
更准确地说，是“检查条件”和“进入睡眠”这两个操作不是原子性的

进程间的相互作用

❖ 信号量:

- 1965年由荷兰学者Dijkstra提出，目前被广泛应用于单处理机和多处理机系统以及计算机网络中。

定义：信号量就是一个带等待队列的计数器，用来管理多个进程/线程对共享资源的访问。

特点：表示资源的实体——是一个与队列有关的整型变量。其值只能通过初始化操作和P、V操作来访问。



初始状态（绿灯 2）表示有 2 个可用资源，**首次 P 操作**（黄灯 1）代表一个进程开始占用资源，**二次 P 操作**（红灯 0）表示资源耗尽，**V 操作**（绿灯 1）则释放资源恢复可用状态。

信号灯的数字变化直观对应信号量值，而火车的通行状态模拟了进程对临界资源的访问控制过程。

信号量的类型：

- **公用信号量**：用于进程间的互斥，初值通常为1；
- **私有信号量**：用于进程间的同步，初值通常为0或n。

进程间的相互作用

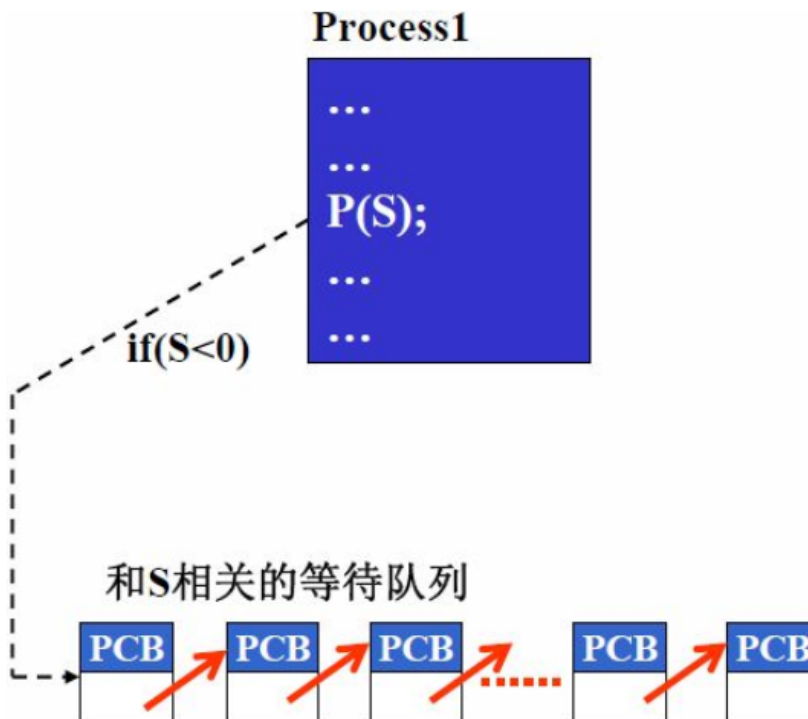
★ 信号量(Semaphore)和P、V操作

❖ P操作

- 荷兰语 “proberen”——“检测” 之意。意味着请求分配一个单位资源。
- 原语操作，表示如下：

P(S): //S为信号量

```
{  
    S = S - 1;  
    if (S < 0)  
    {  
        调用进程被阻塞,  
        进入S的等待队列;  
    }  
}
```



进程间的相互作用

★ 信号量(Semaphore)和P、V操作

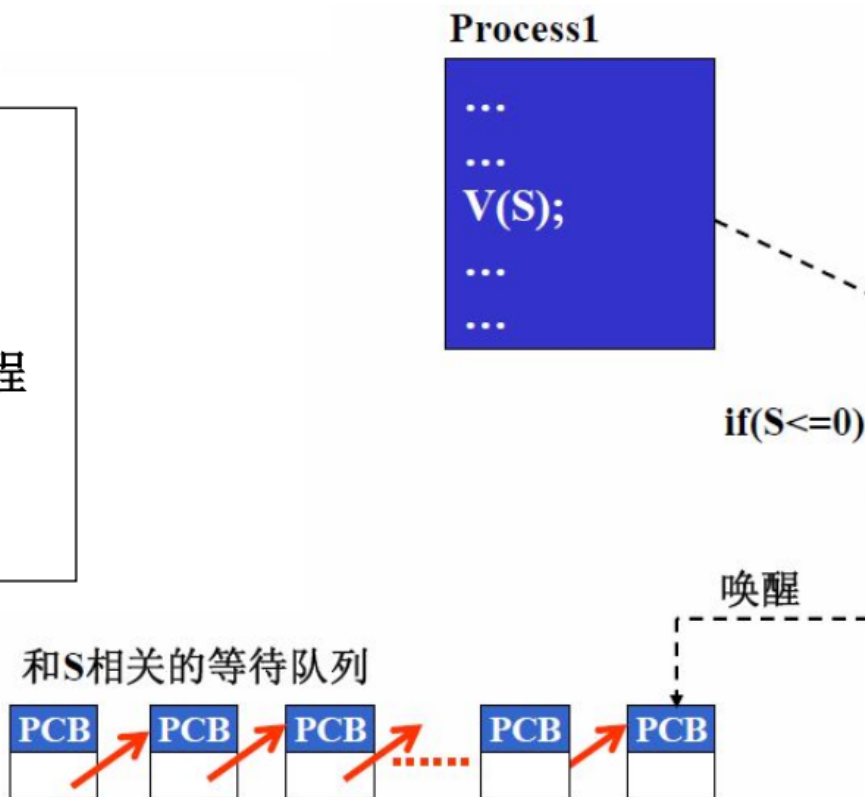
❖ V操作

- V操作：荷兰语“**verhogen**”——“增量”之意，意味着**释放**一个单位资源。
- 原语操作，表示如下：

V(S): //S为信号量

```
{  
    S = S + 1;  
    if (S <= 0)  
    {  
        从S的等待队列中唤醒一个进程  
        使其进入就绪状态;  
    }  
}
```

注意点：V操作在唤醒别人的同时不会阻塞调用进程。



进程间的相互作用

★ 信号量(Semaphore)和P、V操作

信号量的类型：

- **公用信号量**：用于进程间的互斥，初值通常为1；
- **私有信号量**：用于进程间的同步，初值通常为0或n。

1. 公用信号量（用于互斥，Mutex）—— 初值=1，表示共享资源最初有1个可用名额

目的：保护临界区（Critical Section），确保在任何时刻只有一个进程可以访问共享资源（如打印机、共享变量、文件）

进程间的相互作用

★ 信号量(Semaphore)和P、V操作

P操作（申请资源）：

- 进程A执行P(S)： $S = S - 1$
 - 如果 $S \geq 0$ (即 $S=1-1=0$)，说明它成功拿到了钥匙。进程A可以进入临界区使用资源
 - 此时 **S=0** 的含义是：资源已被占用，且暂无进程等待
- 如果进程B在A未退出时也执行P(S)： $S = S - 1$ (即 $S=0-1=-1$)
 - 现在 $S < 0$ ，说明资源已被占用（A在用），且没有空闲名额。进程B被阻塞，放入该信号量的等待队列
 - 此时 **S=-1** 的含义是：有1个进程在等待。**S=-2** 就表示有2个进程在等待，以此类推

V操作（释放资源）：

- 进程A用完资源后，执行V(S)： $S = S + 1$ (即 $S=-1+1=0$)
- 因为 $S \leq 0$ (这里**S=0**)，说明有进程在等待（进程B）。于是，V操作会唤醒等待队列中的进程B
- 进程B被唤醒后，获得CPU，从它之前阻塞的地方（P操作）继续执行，成功进入临界区

进程间的相互作用

★ 信号量(Semaphore)和P、V操作

2. 私有信号量（用于同步，Synchronization）—— **初值=0**，表示最初期望的“同步资源”数量为0

目的： 协调多个进程的执行顺序，确保某些操作必须在其他操作之后发生。典型例子是生产者-消费者问题。

P操作（等待事件发生）：

- **消费者进程**执行 $P(S_product)$ ： $S = S - 1$ (即 $S=0-1=-1$)
- 因为 $S < 0$ 立即成立，消费者进程直接被阻塞：还没产品，请消费者等待
- 此时 **$S=-1$** 的含义是：有1个进程在等待“产品可用”这个事件发生

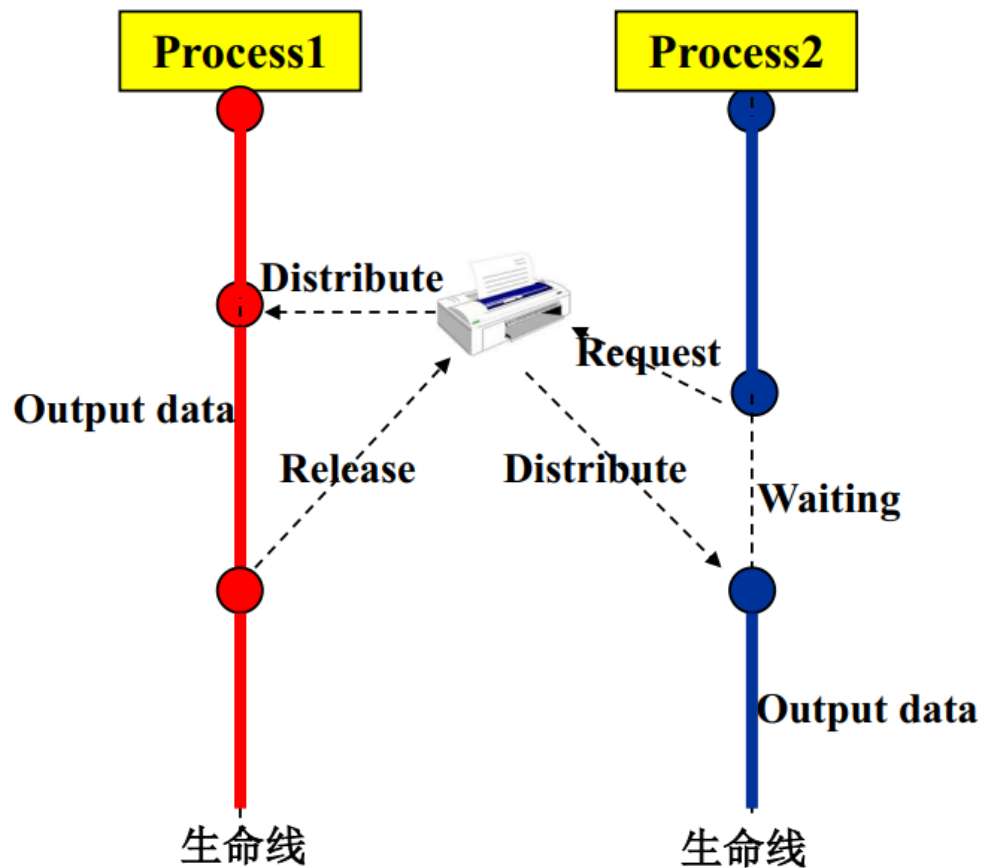
V操作（通知事件发生）：

- **生产者进程**生产了一个产品后，执行 $V(S_product)$ ： $S = S + 1$ (即 $S=-1+1=0$)
- 因为 **$S \leq 0$** ，说明有进程在等待（消费者），V操作会唤醒等待的消费者进程。
- 消费者被唤醒，开始消费产品

关键： V操作是生产者的“通知”，P操作是消费者的“等待”。通过这种机制，保证了“先生产，后消费”的合理顺序

进程间的相互作用

★ 使用P、V操作实现进程互斥



设置信号量:

Semaphore: S; //公用信号量
S=1;

Process1

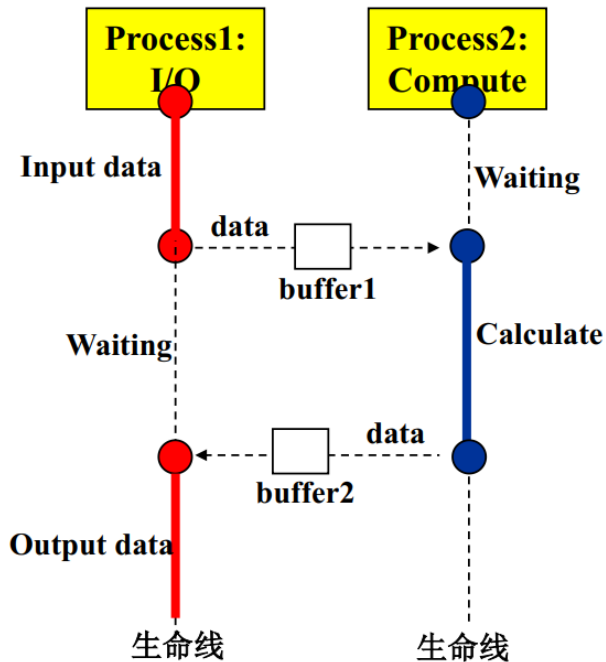
```
{  
    ....  
    P(S);  
    print data;  
    V(S);  
    ....  
}
```

Process2

```
{  
    ....  
    P(S);  
    print data;  
    V(S);  
    ....  
}
```


进程间的相互作用

★ 使用P、V操作实现进程同步



- **S1和S3是空闲信号量**，保护写入操作的互斥
- **S2和S4是数据信号量**，实现同步通信
- **信号量：**
 - **S1 (初值1):** 控制 buffer1 的空闲位置。P(S1)申请写入权，V(S1)释放空闲位
 - **S2 (初值0):** 控制 buffer1 的数据可用性。V(S2)增加一个数据，P(S2)申请读取一个数据
 - **S3 (初值1):** 控制 buffer2 的空闲位置。P(S3)申请写入权，V(S3)释放空闲位
 - **S4 (初值0):** 控制 buffer2 的数据可用性。V(S4)增加一个数据，P(S4)申请读取一个数据

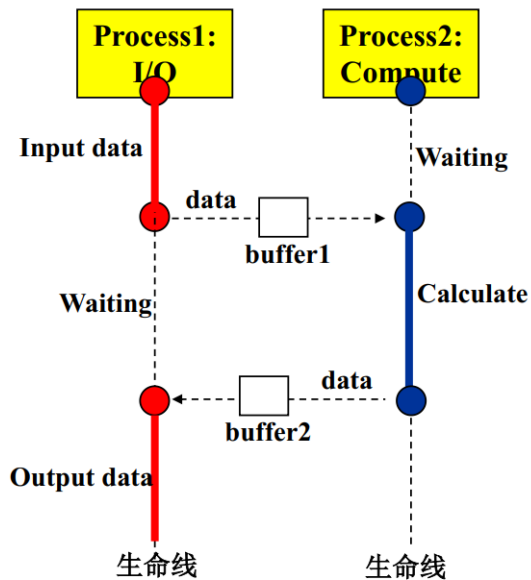
- **Process1 (I/O进程):**
 - 是 buffer1 的生产者 (写入数据)
 - 是 buffer2 的消费者 (读取数据并输出)
- **Process2 (计算进程):**
 - 是 buffer1 的消费者 (读取数据并计算)
 - 是 buffer2 的生产者 (写入计算结果)

进程间的相互作用

★ 使用P、V操作实现进程同步

设置信号量:

$S1=1$; // buffer1 “空” (可写入)
 $S2=0$; // buffer1 “没数据” (需等待)
 $S3=1$; // buffer2 “空” (可写入)
 $S4=0$; // buffer2 “没数据” (需等待)



步骤 1: Process1 输入并写入 Buffer1

- Process1: Input data; -> 准备好数据
- Process1: $P(S1)$;
 - 操作: $S1 = 1 - 1 = 0$
 - 结果: $0 \geq 0$, 成功获得buffer1的写入权, Process1 进入临界区, 准备写buffer1
 - 含义: Buffer1 现在被占用了 (状态: 正在写入)
- Process1: put data to buffer1; -> 将数据写入buffer1
- Process1: $V(S2)$;
 - 操作: $S2 = 0 + 1 = 1$
 - 结果: $1 > 0$, 唤醒正在等待S2的进程 (目前没有, 但信号记录了 “有1个数据可用”)
 - 含义: Buffer1 现在有数据了, 生产了一个 “数据资源”, 这是向Process2发出的信号
- 此时状态: $S1=0, S2=1, S3=1, S4=0$
 - (Buffer1忙/无空位, Buffer1有数据, Buffer2空, Buffer2无数据)

进程间的相互作用

此时信号量:

$S1=0$; // buffer1 “无空位” ; $S2=1$; // buffer1 “有数据” ;

$S3=1$; // buffer2 “空” ; $S4=0$; // buffer2 “没数据”

步骤 2: Process2 从 Buffer1 读取并计算, 然后写入 Buffer2

• Process2: **P(S2)**; (假设Process2现在开始执行或被V(S2)唤醒)

▫ 操作: $S2 = 1 - 1 = 0$

▫ 结果: $0 \geq 0$, 成功获得buffer1的读取权。Process2进入临界区, 准备读buffer1

• Process2: get data from buffer1; calculate; -> 取数据并进行计算

• Process2: **V(S1)**;

▫ 操作: $S1 = 0 + 1 = 1$

▫ 结果: $1 > 0$, 唤醒正在等待S1的进程 (Process1可能下次想写时会在P(S1)等待)

• Process2: **P(S3)**;

▫ 操作: $S3 = 1 - 1 = 0$

▫ 结果: $0 \geq 0$, 成功获得buffer2的写入权。Process2进入临界区, 准备写buffer2

• Process2: put data to buffer2; -> 将计算结果写入buffer2

• Process2: **V(S4)**;

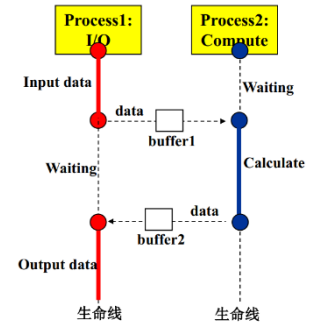
▫ 操作: $S4 = 0 + 1 = 1$

▫ 结果: $1 > 0$, 唤醒正在等待S4的进程 (Process1)

▫ 含义: Buffer2 现在有数据了, 生产了一个“数据资源”, 这是向Process1发出的信号

• 此时状态: $S1=1, S2=0, S3=0, S4=1$

• (Buffer1空, Buffer1无数据, Buffer2忙/无空位, Buffer2有数据)



进程间的相互作用

此时信号量:

$S1=1$; // buffer1 “空” ; $S2=0$; // buffer1 “没数据” ;
 $S3=0$; // buffer2 “无空位” ; $S4=1$; // buffer2 “有数据”

步骤 3: Process1 从 Buffer2 读取并输出

- **Process1: P(S4);**

- 操作: $S4 = 1 - 1 = 0$
- 结果: $0 \geq 0$, 成功获得buffer2的读取权。Process1进入临界区, 准备读buffer2
- 含义: Buffer2 的唯一数据被我取走了

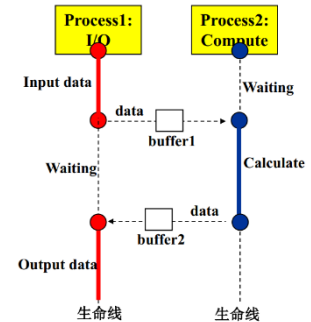
- **Process1: get data from buffer2; output data; -> 取结果并输出**

- **Process1: V(S3);**

- 操作: $S3 = 0 + 1 = 1$
- 结果: $1 > 0$, 唤醒正在等待S3的进程 (Process2可能下次想写时会在P(S3)等待)
- 含义: 我消费完了, Buffer2 现在又空了! 我释放了一个“空闲位置”资源

- **状态恢复: $S1=1, S2=0, S3=1, S4=0$**

- (Buffer1空, Buffer1无数据, Buffer2空, Buffer2无数据) -> **回到初始状态**, 等待下一次循环



进程间的相互作用

★ 进程 1 和进程 2 的生命周期

进程1 (I/O进程) 的生命周期: 不断地从外部设备输入数据, 处理后输出

【生产者角色】 向 Buffer1 写入输入数据

- P(S1): 申请 Buffer1 的**空闲位置** (写入权限)
- 执行IO, 将数据写入Buffer1
- V(S2): 释放一个Buffer1的**数据信号**, 通知进程2 “有数据可读了”

【消费者角色】 从 Buffer2 读取计算结果并输出

- P(S4): 申请一个Buffer2的**数据信号** (等待进程2的计算结果)
- 从Buffer2中读取数据
- V(S3): 释放 Buffer2 的**空闲位置**, 通知进程2 “空位已腾出”
- 将数据输出

进程2 (计算进程) 的生命周期: 不断地处理数据, 然后送出结果

【消费者角色】 从 Buffer1 读取数据进行计算

- P(S2): 申请一个Buffer1的**数据信号** (等待进程1的输入数据)
- 从Buffer1中读取数据
- V(S1): 释放 Buffer1 的**空闲位置**, 通知进程1 “空位已腾出”
- 对数据进行计算

【生产者角色】 向 Buffer2 写入计算结果

- P(S3): 申请 Buffer2 的**空闲位置** (写入权限)
- 将计算结果写入Buffer2
- V(S4): 释放一个Buffer2的**数据信号**, 通知进程1 “结果已就绪”

在这个生产者-消费者模型中, 对缓冲区的“所有权”或“使用权”是随着数据一起传递的

进程间的相互作用

★ 采用信号量机制解决“生产者-消费者问题”

```
#define N n
typedef int semaphore
semaphore mutex=1;    #公用信号量, 互斥锁, 防止多生产者/消费者同时操作缓冲区
semaphore empty=N;    #私有信号量, 同步锁, 空缓冲区数量 (生产者需申请空缓冲区)
semaphore full=0;     #私有信号量, 同步锁, 满缓冲区数量 (消费者需申请满缓冲区)

void producer(void)
{
    int item;
    while(Ture){
        item=produce_item( );
        P(&empty); #申请空缓冲区 (empty减1, 无空则阻塞)
        P(&mutex); #申请互斥锁 (进入临界区)
        insert_item(item);
        V(&mutex); #释放互斥锁 (退出临界区)
        V(&full);  #通知满缓冲区 (full加1, 唤醒消费者)
    }
}

void Consumer(void)
{
    int item;
    while(Ture){
        P(&full);
        P(&mutex);
        item=remove_item( );
        V(&mutex);
        V(&empty);
        consume_item(item);
    }
}
```

进程间的相互作用

使用P、V操作时的注意事项

- ❖ P、V操作（对同一信号量）总是成对出现的；互斥操作时他们处于同一进程中；同步操作时他们处于不同进程中。
- ❖ P、V操作的位置和次序十分重要，使用时注意逻辑关系。

互斥场景——P/V操作在“同一进程”里（以生产者为例）

互斥的核心是“保护共享资源（缓冲区）不被同时操作”，所以P(mutex)和V(mutex)（缓冲区的申请和释放）必须在同一个进程的临界区前后

```
void producer() { while (True) {  
    // ... 生产数据 ...  
    P(&empty);    // 同步操作（和消费者进程的 V(empty) 对应）  
    // ↓ 临界区开始 ↓  
    P(&mutex);    // 申请互斥锁（同一进程内的 P/V 操作）  
    insert_item(item); // 操作共享缓冲区（临界区）  
    V(&mutex);    // 释放互斥锁（同一进程内的 P/V 操作）  
    // ↑ 临界区结束 ↑  
    V(&full);     // 同步操作（和消费者进程的 P(full) 对应） }}
```

进程间的相互作用

使用P、V操作时的注意事项

- ❖ P、V操作（对同一信号量）总是成对出现的；互斥操作时他们处于同一进程中；同步操作时他们处于不同进程中。
- ❖ P、V操作的位置和次序十分重要，使用时注意逻辑关系。

同步场景——P/V操作在“不同进程”里（生产者⇌消费者）

同步的核心是“让进程按顺序执行（生产→消费）”，所以P(empty)/V(empty)、P(full)/V(full)必须分布在不同进程

1. 空缓冲区同步（empty信号量）：

生产者里的P：P(&empty); //生产者生产前，申请一个空缓冲区

消费者里的V：V(&empty); //消费者消费完，释放一个空缓冲区(通知生产者“可以生产了”)

→P(empty)在生产者进程，V(empty)在消费者进程→不同进程的P/V操作

2. 满缓冲区同步（full信号量）：

生产者里的V：V(&full); //生产者生产完，释放一个满缓冲区(通知消费者“有数据了”)

消费者里的P：P(&full); //消费者消费前，申请一个满缓冲区(等待生产者“生产数据”)

→V(full)在生产者进程，P(full)在消费者进程→不同进程的P/V操作

进程间的相互作用

信号量机制的缺点

- ❖ 信号量机制的引入解决了进程同步问题，但信号量的大量同步操作分散在各个进程中不便于管理，使用不当还可能导致系统死锁。

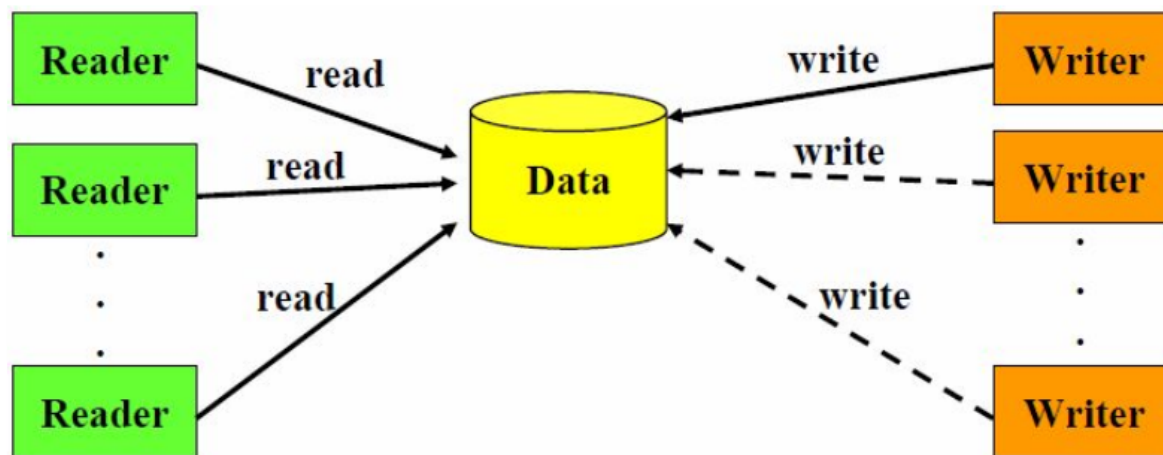
Dijkstra1971年提出一个概念

- ❖ 把所有进程对某一种临界资源的同步操作都集中起来，构成一个所谓的秘书进程。凡要访问该临界资源的进程，都需先报告秘书，由秘书来实现诸进程对同一临界资源的互斥使用。

进程间的相互作用

IPC经典问题——读者写者问题

- ❖ 问题描述：一个数据对象（文件、记录）可以为多个并发进程共享。其中有的进程只需要读其中的内容，我们称为“**读者**”；有的进程负责更新其中内容，我们称为“**写者**”。规定：“读者”可以同时读取共享数据对象；“写者”不能和其它任何进程同时访问共享数据对象。



进程间的相互作用

★ “读者 - 写者问题” 的同步机制

❖ 分析：

➤ 分析进程行为：

读进程的行为：系统中会有多个读进程同时访问共享数据，不过我们可以将他们的行为合并，只需要区分第一个进入的读进程(占有临界资源)和最后一个离开的读进程(释放临界资源)。

因此需要设置一个计数器**readnum**来记录读进程的数目。

➤ 确定临界资源：

Data; readnum

➤ 确定同步和互斥关系：

- ① “读者——读者”：互斥访问**readnum**
- ② “读者——写者”：互斥访问**Data**
- ③ “写者——写者”：互斥访问**Data**

进程间的相互作用

★ “读者 - 写者问题” 的信号量实现

设置信号量:

`int readnum=0;` //计数, 用于记录读者的数目

`semaphore mutex=1, write=1;` //公用信号量, 用于互斥: `mutex`用于对 `readnum`互斥; `write`用于写者之间以及写者和读者之间互斥

读者进程:

```
P(mutex); //对readnum互斥
readnum ++; //读者数目加1
if (readnum==1) //第一个读进程
    P(write); //申请使用data资源
V(mutex); //释放readnum
reading;
P(mutex); //对readnum互斥
readnum --;
if (readnum==0) //最后一个读进程
    V(write); //释放data资源
V(mutex); //释放readnum
```

写者进程:

```
P(write); //申请使用data资源
writing;
V(write); //释放data资源
```

进程间的相互作用

★ “读者 - 写者问题” 的信号量实现——以2 个读者(R1、R2)访问 Data为例

读者 R1 进入, 执行:

```
P(mutex);      // 申请 mutex(值1→0), 保护 readnum  
readnum ++;    // readnum = 1 (记录 R1 进入)  
if (readnum == 1) {  
P(write); } // 申请 write(值1→0), 抢占写者访问权  
V(mutex); // 释放 mutex(值0→1), 允许其他读者修改 readnum  
reading;      // R1 开始读 Data(并发读, 无需锁)
```

读者 R2 进入, 执行:

```
P(mutex);      // 申请 mutex(值1→0), 保护 readnum  
readnum ++;    // readnum = 2 (记录 R2 进入)  
if (readnum == 1) {  
P(write); } // 此时 readnum = 2 ≠ 1, 不执行 P(write)  
V(mutex); // 释放 mutex(值0→1), 允许其他读者修改 readnum  
reading;      // R2 开始读 Data(并发读, 无需锁)
```

进程间的相互作用

★ “读者 - 写者问题” 的信号量实现——以2 个读者(R1、R2)访问 Data为例

读者 R1 离开, 执行:

```
P(mutex);      // 申请 mutex(值1→0), 保护 readnum  
readnum --;    // readnum = 1 (记录 R1 离开, R2还在)  
if (readnum == 0) {  
V(write); } // 此时 readnum = 1 ≠ 0, 不执行 V(write)  
V(mutex); // 释放 mutex(值0→1), 允许其他读者修改 readnum  
// R1 读完, 离开
```

读者 R2 离开, 执行:

```
P(mutex);      // 申请 mutex(值1→0), 保护 readnum  
readnum --;    // readnum = 0 (记录 R2 离开, 此时所有读者离开)  
if (readnum == 0) {  
V(write); } // 释放 write (值1→1) , 允许写者访问 Data  
V(mutex); // 释放 mutex(值0→1), 允许其他读者修改 readnum  
// R2 读完, 离开
```

进程间的相互作用

IPC经典问题——理发师问题



问题描述:

- 理发店有一位理发师，1 把理发椅和 n 把用来等候理发的椅子；
- 如果没有顾客，则理发师在理发椅上睡觉；
- 顾客来时，如理发师空闲则理发，否则如有空椅则坐等，没有空椅则离开；
- 编写程序实现理发师和顾客程序。

进程间的相互作用

★ “理发师问题” 的信号量实现

设置信号量:

Semaphore customers=0,barbers=0,mutex=1;

//customers表示顾客数量, barbers 表示理发师状态

int waiting=0; //等待人数

Process barber

```
{
  While(1)
  {
    P(customers );//检查是否有顾客
    P(mutex);
    waiting=waiting-1;
    V(mutex);
    V(barbers); //向顾客发信号
    Cut hair();
  }
}
```

Process customer

```
{
  P(mutex);
  If (waiting<CHIRS) then
  {
    waiting=waiting+1;
    V(mutex);
    V(customers); //向理发师发信号
    P(barbers); //接收理发师信号
    Get_haircut();
  }
  else
  {
    V(mutex);
  }
}
```


进程间的相互作用

IPC经典问题——哲学家问题

- ❖ 五个哲学家围坐在一圆桌旁，桌中央有一盘通心粉，每人面前有一只空盘子，每两人之间放一只叉子。每个哲学家的行为是思考，感到饥饿，然后吃通心粉。为了吃通心粉，每个哲学家必须拿到两只叉子，并且每个人只能直接从自己的左边或右边去取叉子。



进程间的相互作用

★ 用P、V操作解决哲学家进餐问题

// 信号量数组：每个叉子对应一个信号量，初始值为1（可用）

```
semaphore fork[5] = {1, 1, 1, 1, 1};
```

// 哲学家 i 的活动

```
void philosopher(int i) {
```

```
    while (true) {
```

```
        思考;
```

```
        P(fork[i]);           // 拿左边的叉子（申请左叉资源）
```

```
        P(fork[(i+1) % 5]);    // 拿右边的叉子（申请右叉资源）
```

```
        进餐;
```

```
        V(fork[i]);           // 放下左边的叉子（释放左叉资源）
```

```
        V(fork[(i+1) % 5]);    // 放下右边的叉子（释放右叉资源）
```

```
    }
```

```
}
```



可能出现死锁:

当5个哲学家同时
执行P(fork[i])

（都拿到左边的
叉子），此时所有
fork 信号量的
值都为0。接下来
每个哲学家会执
行P(fork[(i+1) %
5])，但所有右边
的叉子都已被占
用，导致所有哲
学家阻塞，形成
死锁。

进程间的相互作用

★ 用P、V操作解决哲学家进餐问题 —— 改进（破坏循环等待）

```
// 信号量数组：每个叉子对应一个信号量，初始值为1（可用）
semaphore fork[5] = {1, 1, 1, 1, 1};
// 哲学家 i 的活动
void philosopher(int i) {
    while (true) {
        思考;
        int left = i;
        int right = (i + 1) % 5;
        if (left < right) { // 关键：先拿编号小的叉子，再拿编号大的叉子
            P(fork[left]);    // 先拿小编号叉子
            P(fork[right]);   // 再拿大编号叉子
        } else {
            P(fork[right]);   // 小编号在右侧时，先拿右侧
            P(fork[left]);    // 再拿左侧
        }
        进餐;
        // 释放叉子（顺序无关）
        V(fork[left]);
        V(fork[right]);    }}

```

避免死锁：

通过强制 “先获取小编号叉子，再获取大编号叉子” 的规则，消除环形等待的可能。

例如：哲学家 4 的左边是叉子 4，右边是叉子 0 ($0 < 4$)，因此会先尝试拿叉子 0，再拿叉子 4。当哲学家 0 和 4 同时竞争时，只有一个能拿到叉子 0，另一个会阻塞，避免了 “每个哲学家都持有一把叉子并等待另一把” 的死锁局面。

进程间的相互作用

★ 用P、V操作解决哲学家进餐问题 —— 改进（破坏“占有并等待”）

```
// 全局信号量mutex，初始值为1，控制对叉子的原子操作
// 叉子状态数组forks[5]，1表示可用，0表示被占用
```

哲学家 i 的活动:

while True:

 思考

P(mutex) // 进入临界区，确保原子性检查和获取叉子

 if 叉子i和叉子(i+1)%5都可用:

 拿起叉子i (forks[i] = 0)

 拿起叉子(i+1)%5 (forks[(i+1)%5] = 0)

V(mutex) // 释放临界区

 进餐

P(mutex) // 进入临界区释放叉子

 放下叉子i (forks[i] = 1)

 放下叉子(i+1)%5 (forks[(i+1)%5] = 1)

V(mutex)

else:

 V(mutex) // 若无法获取，直接释放临界区，不占有任何叉子

避免死锁:

通过全局锁确保“检查叉子可用性”和“获取叉子”的原子性，避免哲学家持有一把叉子并等待另一把的情况，从而破坏“占有并等待”条件。

本质：要么同时获得两把叉子，要么一把都不拿

进程间的相互作用

★ 用P、V操作解决哲学家进餐问题 —— 改进（限制同时拿叉子的哲学家数量）

```
// 全局信号量room, 初始值为4, 限制同时拿叉子的哲学家数量
// 每个叉子对应一个信号量fork[5], 初始值为1
```

哲学家 i 的活动:

```
while True:
```

```
    思考
```

```
    P(room) // 进入房间, 最多4人同时尝试拿叉子
```

```
    P(fork[i]) // 拿左叉
```

```
    P(fork[(i+1)%5]) // 拿右叉
```

```
    进餐
```

```
    V(fork[i]) // 放左叉
```

```
    V(fork[(i+1)%5]) // 放右叉
```

```
    V(room) // 离开房间, 允许新的哲学家进入
```

避免死锁:

通过限制并发拿叉子的人数（小于哲学家总数），确保至少有一个哲学家能同时获取两把叉子，避免所有人都卡住的死锁局面。

关键： room<0时等待

进程间的相互作用

管程思想

❖ 集中和封装针对一个共享资源的所有访问，包括所需的同步操作

管程是由一组**局部的变量**、**对局部变量进行操作的一组过程**以及**对局部变量进行初始化的语句序列**构成的一个**软件模块**



信号量机制的缺点：信号量机制的引入解决了进程同步问题，但信号量的**大量同步操作分散在各个进程中**不便于管理，使用不当还可能导致系统死锁。

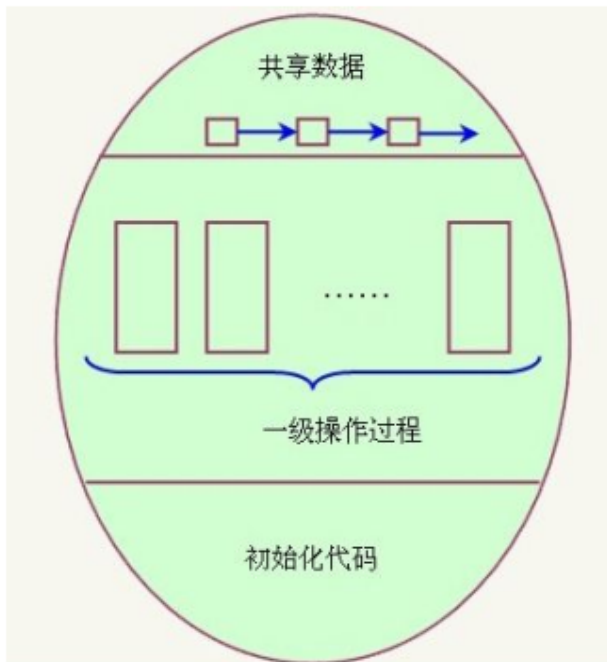
管程结构：

```
Monitor monitor_name; // 管程名
variable declarations; // 共享变量说明 (仅管程内可访问)
void Entry P1(...) { ... } // 对数据结构操作的函数, 入口函数
// (进程通过调用这些函数访问共享资源)
void Entry P2(...) { ... }
...
void Entry Pn(...) { ... }
{initialization code; } // 初始化代码
```

进程间的相互作用

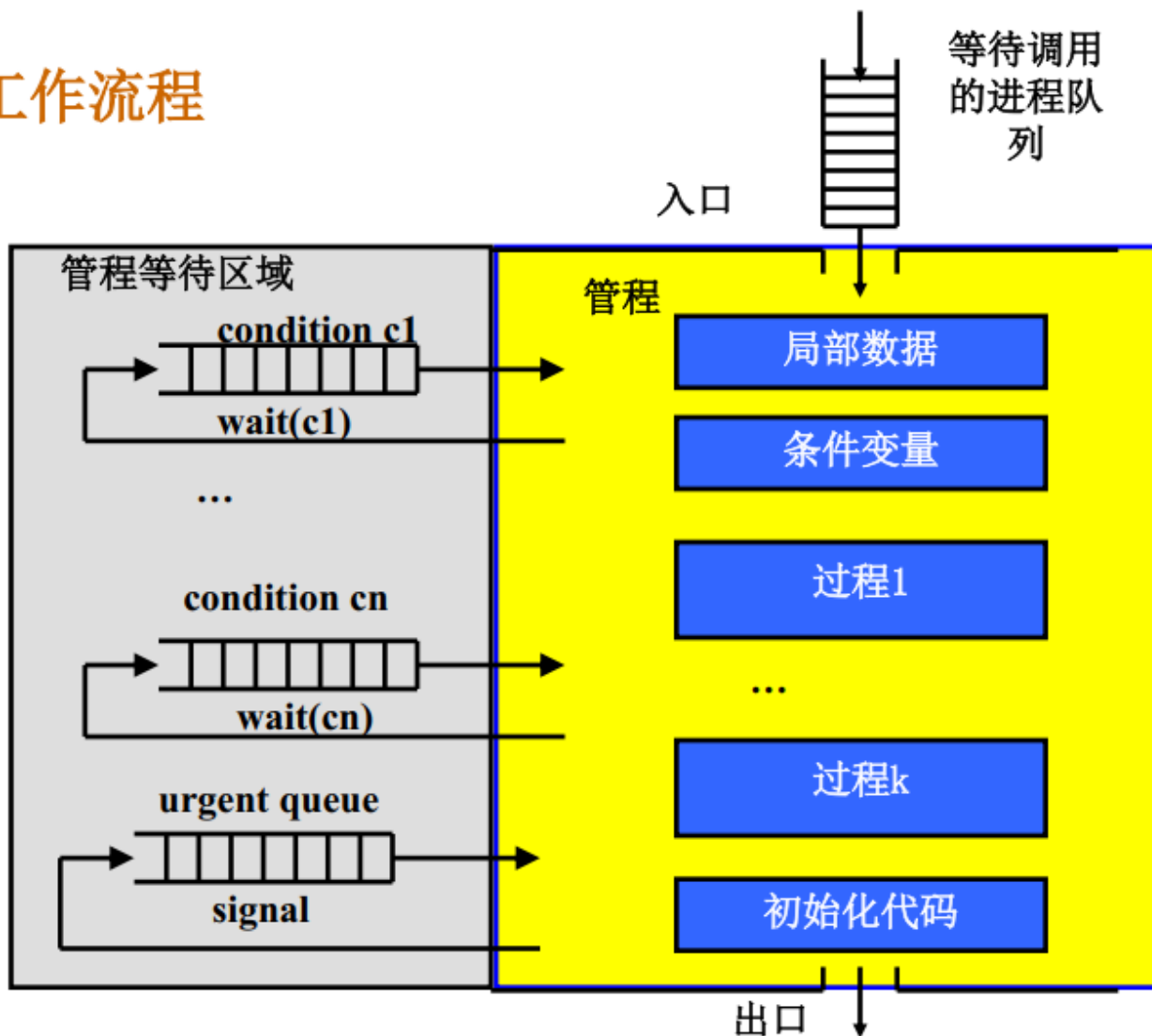
管程的基本特性

- ❖ 局部于管程的数据只能被局部于管程内的函数所访问。
- ❖ 一个进程只有通过调用管程内的函数才能进入管程访问共享数据。
- ❖ 每次仅允许一个进程在管程内执行某个函数。
- ❖ 由于管程是一个语言成分，所以管程的互斥访问完全由编译程序在编译时自动添加上，无需程序员关心，而且保证正确。



进程间的相互作用

管程的工作流程



进程间的相互作用

★ 用管程解决哲学家进餐问题

Monitor Dining Philosophers {

int fork[5] = {0, 0, 0, 0, 0}; // 1. 共享变量: 表示叉子状态 (0=空闲, 1=被占用)

Condition cond[5]; // 2. 条件变量: 当叉子被占用时, 哲学家等待

void Entry pickup(int i) { // 3. 入口函数: 拿左边和右边叉子

int left = i, right = (i+1)%5; // i 是哲学家编号 (0~4), 左叉=i, 右叉=(i+1)%5

if (fork[left] == 1 || fork[right] == 1) { // 检查叉子是否可用

cond[i].wait(); // 叉子被占用, 等待

} else {

fork[left] = 1; fork[right] = 1; } } // 叉子可用, 标记为占用

void Entry putdown(int i) { // 4. 入口函数: 放下叉子

int left = i, right = (i+1)%5;

fork[left] = 0; fork[right] = 0; // 标记叉子为空闲

cond[(i-1+5) % 5].signal(); // 唤醒等待的哲学家 (左右)

cond[(i+1) % 5].signal(); }

{ for (int i=0; i<5; i++) fork[i] = 0; } } // 5. 初始化代码: 初始叉子均为空闲

进程间的相互作用

★ 用管程解决哲学家进餐问题

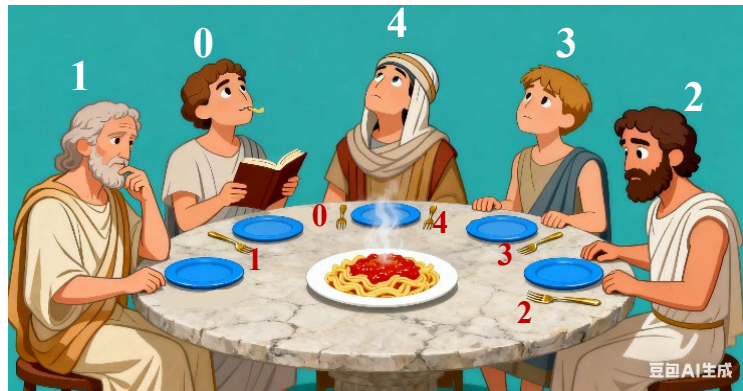
封装性：代码中定义了Monitor Dining Philosophers这样一个管程结构，将共享变量（fork数组）、条件变量（cond数组）以及对共享资源操作的过程（pickup和putdown函数）都封装在管程内部。外部代码要访问这些共享资源，只能通过管程提供的入口函数（pickup和putdown），保证了对共享资源访问的集中控制。

互斥性：管程本身保证了在同一时刻，最多只有一个进程能够进入管程内的临界区执行代码。当一个哲学家在执行pickup或者putdown函数时，其他哲学家无法同时进入管程执行这两个函数，避免了多个进程同时操作共享资源（叉子）而导致的混乱。

同步机制：通过条件变量cond和相关的wait、signal操作实现同步。当哲学家发现需要的叉子被占用时（ $\text{fork}[\text{left}] == 1 \parallel \text{fork}[\text{right}] == 1$ ），会执行 $\text{cond}[i].\text{wait}()$ ，将自己阻塞并释放管程的互斥权，让其他进程有机会进入管程；当有哲学家放下叉子后，执行 $\text{cond}[(i - 1 + 5) \% 5].\text{signal}()$ 和 $\text{cond}[(i + 1) \% 5].\text{signal}()$ ，唤醒等待相应叉子的哲学家，使它们有机会再次尝试获取叉子。

进程间的相互作用

★ 用管程解决哲学家进餐问题 ——整体协作流程（示例）



- 哲学家 0 想进餐：

调用 `pickup(0)` → 检查左右叉子（0 和 1）均空闲 → 标记为占用 → 开始进餐

- 哲学家 1 想进餐：

调用 `pickup(1)` → 右叉子（2）空闲，但左叉子（1）被哲学家 0 占用 → 进入 `cond[1]` 队列等待

- 哲学家 0 吃完放叉子：

调用 `putdown(0)` → 释放叉子 0 和 1 → 唤醒 `cond[1]` 和 `cond[4]` 队列中的哲学家 → 哲学家 1 被唤醒并重新检查叉子状态 → 获得叉子 1 和 2 → 开始进餐

- 哲学家 2 想进餐：

调用 `pickup(2)` → 左叉子（2）已被哲学家 1 占用 → 进入 `cond[2]` 队列等待

- 哲学家 1 吃完放叉子：

调用 `putdown(1)` → 释放叉子 1 和 2 → 唤醒 `cond[0]` 和 `cond[2]` 队列中的哲学家 → 哲学家 2 被唤醒并重新检查叉子状态 → 获得叉子 2 和 3 → 开始进餐

同步、互斥建立在**沟通通信**的基础上
传递 状态 和 控制 信息

论述：

1. **总结：** 进程同步与互斥的实现方式，并思考实现进程同步互斥、通信的各种方法的设计初衷和使用限制是什么？操作系统和用户分别负责什么任务？
2. **总结：** 经典的IPC问题
3. **调研：** (1) 由多道进程协作想到XX智能
(2) 实际操作系统如何实现进程的同步与互斥？